07. 네트워킹 및 P2P 통신 구현 분석

개요

Lighthouse의 네트워킹 시스템은 Ethereum 2.0 합의 프로토콜의 핵심 구성 요소로, 분산된 검증자들 간의 효율적이고 안전한 통신을 담당합니다. 이 문서에서는 `lighthouse_network` 모듈의 구조와 구현을 상세히 분석하여, P2P 네트워킹, Gossipsub 프로토콜, 피어 관리, 그리고 RPC 통신의 실제 동작 원리를 이해해보겠습니다.

전체 네트워킹 아키텍처

핵심 컴포넌트 구조

Lighthouse의 네트워킹 시스템은 다음과 같은 계층적 구조로 구성됩니다:

```
// beacon_node/lighthouse_network/src/service/mod.rs
pub struct Network<E: EthSpec> {
swarm: libp2p::swarm::Swarm<Behaviour<E>>,
network_globals: Arc<NetworkGlobals<E>>,
enr_fork_id: EnrForkId,
network_dir: PathBuf,
fork_context: Arc<ForkContext>,
score_settings: PeerScoreSettings<E>,
update_gossipsub_scores: tokio::time::Interval,
gossip_cache: GossipCache,
```

```
local_peer_id: PeerId,
}
```

Behaviour 구조체 - 네트워크 동작 정의

```
#[derive(NetworkBehaviour)]
pub(crate) struct Behaviour<E>
where
E: EthSpec,
{
// 연결 제한 관리
pub  connection_limits: libp2p::connection_limits::Behaviour,
// 피어 평판 및 상태 관리
pub  peer_manager: PeerManager<E>,
// Ethereum 2.0 RPC 프로토콜
pub  eth2_rpc: RPC<AppRequestId, E>,
// 피어 발견 프로토콜
pub  discovery: Discovery<E>,
// 피어 식별 프로토콜
pub  identify: identify::Behaviour,
// UPnP 포트 매핑
pub  upnp: Toggle<Upnp>,
// Gossipsub 메시징
```

```
pub  gossipsub: Gossipsub,

}
```

Gossipsub 프로토콜 구현

Gossipsub 설정 및 초기화

Lighthouse는 Ethereum 2.0에 최적화된 Gossipsub 구성을 사용합니다:

```
// Gossipsub 타입 정의

pub  type Gossipsub = gossipsub::Behaviour<SnappyTransform, SubscriptionFilter>;

pub  type SubscriptionFilter =

gossipsub::MaxCountSubscriptionFilter<gossipsub::WhitelistSubscriptionFilter>;



// Gossipsub 초기화 과정

let (gossipsub, update_gossipsub_scores) = {

let  thresholds = lighthouse_gossip_thresholds();

// 피어 스코어링 매개변수 설정

let  params = score_settings.get_peer_score_params(

active_validators,

&thresholds,

&enr_fork_id,

current_slot,

)?;
```

```
// 스코어 업데이트 간격 설정
let update_gossipsub_scores = tokio::time::interval(params.decay_interval);
// Snappy 압축 변환기 설정
let snappy_transform = SnappyTransform::new(
spec.max_payload_size as usize,
spec.max_compressed_len()
);
let mut gossipsub = Gossipsub::new_with_subscription_filter_and_transform(
MessageAuthenticity::Anonymous,
gs_config.clone(),
gossipsub_metrics,
filter,
snappy_transform,
)?;
// 피어 스코어링 적용
gossipsub.with_peer_score(params, thresholds)?;
(gossipsub, update_gossipsub_scores)
};
```

토픽 관리 시스템

GossipKind - 메시지 유형 정의

```
// beacon_node/lighthouse_network/src/types/topics.rs
```

```rust
#[derive(Clone, Debug, Serialize, Deserialize, PartialEq, Eq, Hash, AsRefStr)]
#[strum(serialize_all = "snake_case")]
pub  enum GossipKind {
/// 비콘 블록 발행
BeaconBlock,
/// 집계된 증명 및 증명서 발행
BeaconAggregateAndProof,
/// 블롭 사이드카 발행 (Deneb 포크)
BlobSidecar(u64),
/// 데이터 컬럼 사이드카 발행 (Fulu 포크)
DataColumnSidecar(DataColumnSubnetId),
/// 특정 서브넷의 원시 증명
#[strum(serialize = "beacon_attestation")]
Attestation(SubnetId),
/// 자발적 탈퇴
VoluntaryExit,
/// 블록 제안자 슬래싱
ProposerSlashing,
/// 증명자 슬래싱
AttesterSlashing,
/// 부분 집계된 동기화 위원회 서명
SignedContributionAndProof,
/// 특정 서브넷의 비집계 동기화 위원회 서명
```

```
#[strum(serialize = "sync_committee")]
SyncCommitteeMessage(SyncSubnetId),
/// 출금 주소 변경
BlsToExecutionChange,
/// 라이트 클라이언트 최종성 업데이트
LightClientFinalityUpdate,
/// 라이트 클라이언트 낙관적 업데이트
LightClientOptimisticUpdate,
}
```

토픽 구독 관리

```
// 핵심 토픽 구독 함수
pub fn core_topics_to_subscribe<E: EthSpec>(
fork_name: ForkName,
opts: &TopicConfig,
spec: &ChainSpec,
) -> Vec<GossipKind> {
let mut topics = vec![
GossipKind::BeaconBlock,
GossipKind::BeaconAggregateAndProof,
GossipKind::VoluntaryExit,
GossipKind::ProposerSlashing,
```

```
GossipKind::AttesterSlashing,
];

// 모든 서브넷 구독 옵션
if opts.subscribe_all_subnets {
for i in 0..spec.attestation_subnet_count {
topics.push(GossipKind::Attestation(i.into()));
}
}

// Altair 포크 이후 토픽들
if fork_name.altair_enabled() {
topics.push(GossipKind::SignedContributionAndProof);
if opts.subscribe_all_subnets {
for i in 0..E::SyncCommitteeSubnetCount::to_u64() {
topics.push(GossipKind::SyncCommitteeMessage(i.into()));
}
}
// 라이트 클라이언트 서버 기능
if opts.enable_light_client_server {
topics.push(GossipKind::LightClientFinalityUpdate);
topics.push(GossipKind::LightClientOptimisticUpdate);
}
```

```
}

// Deneb 포크의 블롭 사이드카
if fork_name.deneb_enabled() && !fork_name.fulu_enabled() {
for i in 0..spec.blob_sidecar_subnet_count(fork_name) {
topics.push(GossipKind::BlobSidecar(i));
}
}

// Fulu 포크의 데이터 컬럼 사이드카
if fork_name.fulu_enabled() {
if opts.subscribe_all_data_column_subnets {
for i in 0..spec.data_column_sidecar_subnet_count {
topics.push(GossipKind::DataColumnSidecar(i.into()));
}
} else {
for subnet in &opts.sampling_subnets {
topics.push(GossipKind::DataColumnSidecar(*subnet));
}
}
}

topics
```

```
}
```

메시지 처리 및 전파

토픽 인코딩 및 디코딩

```
impl std::fmt::Display for GossipTopic {
fn  fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
let  encoding = match  self.encoding {
GossipEncoding::SSZSnappy => SSZ_SNAPPY_ENCODING_POSTFIX,
};

let  kind = match  self.kind {
GossipKind::BeaconBlock => BEACON_BLOCK_TOPIC.into(),
GossipKind::Attestation(index) => {
format!("{}{}", BEACON_ATTESTATION_PREFIX, *index)
},
GossipKind::SyncCommitteeMessage(index) => {
format!("{}{}", SYNC_COMMITTEE_PREFIX_TOPIC, *index)
},
// ... 기타 토픽들
};
write!(
f,
```

```
"/{}/{}/{}/{}",
TOPIC_PREFIX, // "eth2"
hex::encode(self.fork_digest),
kind,
encoding  // "ssz_snappy"
)
}
}
```

네트워크 이벤트 처리

```
// 네트워크에서 발생하는 이벤트 타입들
#[derive(Debug)]
pub  enum NetworkEvent<E: EthSpec> {
/// 아웃바운드 피어 연결 성공
PeerConnectedOutgoing(PeerId),
/// 인바운드 피어 연결 성공
PeerConnectedIncoming(PeerId),
/// 피어 연결 해제
PeerDisconnected(PeerId),
/// RPC 요청 실패
RPCFailed {
app_request_id: AppRequestId,
peer_id: PeerId,
```

```
error: RPCError,
},
/// 수신된 RPC 요청
RequestReceived {
peer_id: PeerId,
inbound_request_id: InboundRequestId,
request_type: RequestType<E>,
},
/// 수신된 RPC 응답
ResponseReceived {
peer_id: PeerId,
app_request_id: AppRequestId,
response: Response<E>,
},
/// Gossipsub 메시지 수신
PubsubMessage {
id: MessageId,
source: PeerId,
topic: TopicHash,
message: PubsubMessage<E>,
},
/// 새로운 리스닝 주소
NewListenAddr(Multiaddr),
```

```
/// 리스너가 없음
ZeroListeners,
/// 피어의 커스터디 그룹 수 업데이트
PeerUpdatedCustodyGroupCount(PeerId),
}
```

RPC 프로토콜 구현

RPC 시스템 구조

```
// beacon_node/lighthouse_network/src/rpc/mod.rs
/// RPC 이벤트 - Lighthouse에서 전송
#[derive(Debug, Clone)]
pub  enum RPCSend<Id, E: EthSpec> {
/// Lighthouse에서 보낸 요청 (아웃바운드 연결)
Request(Id, RequestType<E>),
/// Lighthouse에서 보낸 응답 (인바운드 연결)
Response(SubstreamId, RpcResponse<E>),
/// 연결 종료 요청
Shutdown(Id, GoodbyeReason),
}

/// RPC 이벤트 - 외부에서 수신
```

```rust
#[derive(Debug, Clone)]
pub  enum RPCReceived<Id, E: EthSpec> {
/// 외부에서 받은 요청 (인바운드 서브스트림)
Request(InboundRequestId, RequestType<E>),
/// 외부에서 받은 응답 (아웃바운드 연결)
Response(Id, RpcSuccessResponse<E>),
/// 요청 완료 표시
EndOfStream(Id, ResponseTermination),
}
```

주요 RPC 메서드들

RPC 프로토콜은 다음과 같은 핵심 메서드들을 제공합니다:

1. **Status**: 피어 간 상태 정보 교환
2. **Goodbye**: 연결 종료 알림
3. **Ping/Pong**: 연결 상태 확인
4. **MetaData**: 피어 메타데이터 요청
5. **BeaconBlocksByRange**: 범위별 블록 요청
6. **BeaconBlocksByRoot**: 루트별 블록 요청
7. **LightClientBootstrap**: 라이트 클라이언트 부트스트랩

피어 관리 시스템

PeerManager 구조

```rust
// beacon_node/lighthouse_network/src/peer_manager/mod.rs

pub struct PeerManager<E: EthSpec> {

/// 네트워크 글로벌 상태 접근을 위한 저장소

network_globals: Arc<NetworkGlobals<E>>,

/// PeerManager가 생성 대기 중인 이벤트 큐

events: SmallVec<[PeerManagerEvent; 16]>,

/// Ping 대기 중인 인바운드 연결 피어들

inbound_ping_peers: HashSetDelay<PeerId>,

/// Ping 대기 중인 아웃바운드 연결 피어들

outbound_ping_peers: HashSetDelay<PeerId>,

/// Status 대기 중인 피어들

status_peers: HashSetDelay<PeerId>,

/// 목표 피어 수

target_peers: usize,

/// 다이얼 대기 중인 피어들

peers_to_dial: Vec<Enr>,

// ... 기타 필드들

}
```

피어 연결 관리

연결 제한 설정

```rust
let connection_limits = {
```

```
let  limits = libp2p::connection_limits::ConnectionLimits::default()
.with_max_pending_incoming(Some(5))
.with_max_pending_outgoing(Some(16))
.with_max_established_incoming(Some(
(config.target_peers as f32
* (1.0 + PEER_EXCESS_FACTOR - MIN_OUTBOUND_ONLY_FACTOR))
.ceil() as u32,
))
.with_max_established_outgoing(Some(
(config.target_peers as f32 * (1.0 + PEER_EXCESS_FACTOR)).ceil() as u32,
))
.with_max_established(Some(
(config.target_peers as f32 * (1.0 + PEER_EXCESS_FACTOR + PRIORITY_PEER_EXCESS))
.ceil() as u32,
))
.with_max_established_per_peer(Some(1));


libp2p::connection_limits::Behaviour::new(limits)
};
```

피어 스코어링 시스템

Lighthouse는 피어의 행동을 평가하여 신뢰도를 관리합니다:

```rust
// 피어 액션 타입들
pub enum PeerAction {
/// 치명적 오류 - 즉시 차단
Fatal,
/// 낮은 허용 오차 오류
LowToleranceError,
/// 중간 허용 오차 오류
MidToleranceError,
/// 높은 허용 오차 오류
HighToleranceError,
}

// 스코어 업데이트 결과
pub enum ScoreUpdateResult {
/// 피어가 차단됨
Banned,
/// 스코어가 업데이트됨
ScoreUpdated,
/// 피어를 찾을 수 없음
PeerUnknown,
}
```

피어 발견 (Discovery) 시스템

Discovery 구조 및 기능

```
// beacon_node/lighthouse_network/src/discovery/mod.rs
pub struct Discovery<E: EthSpec> {
/// Discv5 프로토콜 인스턴스
discv5: Discv5,
/// 네트워크 글로벌 상태
network_globals: Arc<NetworkGlobals<E>>,
/// 진행 중인 쿼리들
active_queries: FuturesUnordered<QueryFuture>,
/// 서브넷 발견 쿼리들
subnet_queries: VecDeque<SubnetQuery>,
/// ENR 캐시
enr_cache: LruCache<NodeId, Enr>,
// ... 기타 필드들
}
```

서브넷 기반 피어 발견

```
#[derive(Debug, Clone, PartialEq)]
enum QueryType {
/// 서브넷 피어 검색
```

```
Subnet(Vec<SubnetQuery>),
/// 일반적인 피어 검색
FindPeers,
}

#[derive(Clone, PartialEq)]
struct SubnetQuery {
subnet: Subnet,
min_ttl: Option<Instant>,
retries: usize,
}
```

ENR (Ethereum Node Record) 관리

ENR은 노드의 네트워크 정보와 능력을 담고 있는 서명된 레코드입니다:

```
// ENR 키 상수들
pub const ETH2_ENR_KEY: &str = "eth2";
pub const ATTESTATION_BITFIELD_ENR_KEY: &str = "attnets";
pub const SYNC_COMMITTEE_BITFIELD_ENR_KEY: &str = "syncnets";
pub const PEERDAS_CUSTODY_GROUP_COUNT_ENR_KEY: &str = "custody_group_count";

// ENR 확장 트레이트
pub trait EnrExt {
```

```
/// Ethereum 2.0 관련 데이터 추출
fn eth2(&self) -> Result<EnrForkId, &'static str>;
/// 증명 서브넷 비트필드 추출
fn attestation_bitfield<E: EthSpec>(&self) -> Result<EnrAttestationBitfield<E>,
/// 동기화 위원회 서브넷 비트필드 추출
fn sync_committee_bitfield<E: EthSpec>(&self) -> Result<EnrSyncCommitteeBitfiel
}
```

네트워크 초기화 및 시작 과정

네트워크 서비스 생성

```
impl<E: EthSpec> Network<E> {
pub async fn new(
executor: task_executor::TaskExecutor,
mut ctx: ServiceContext<'_>,
custody_group_count: u64,
) -> Result<(Self, Arc<NetworkGlobals<E>>), String> {
// 1. 로컬 키페어 로드
let local_keypair = utils::load_private_key(&config);
// 2. 신뢰할 수 있는 피어 설정
let trusted_peers: Vec<PeerId> = config
.trusted_peers
```

```
.iter()
.map(|x| PeerId::from(x.clone()))
.collect();
// 3. ENR 생성 또는 로드
let  enr = crate::discovery::enr::build_or_load_enr::<E>(
local_keypair.clone(),
&config,
&ctx.enr_fork_id,
&ctx.chain_spec,
)?;
// 4. 메타데이터 구성
let  meta_data = utils::load_or_build_metadata(&config.network_dir, advertised_c
// 5. 네트워크 글로벌 상태 생성
let  network_globals = Arc::new(NetworkGlobals::new(
enr,
meta_data,
trusted_peers,
config.disable_peer_scoring,
config.clone(),
ctx.chain_spec.clone(),
));
// 6. Gossipsub 설정
let (gossipsub, update_gossipsub_scores) = {
```

```
// Gossipsub 구성 및 초기화

// ... (앞서 설명한 코드)

};

// 7. 각 컴포넌트 초기화

let eth2_rpc = RPC::new(/* ... */);

let discovery = Discovery::new(/* ... */).await?;

let identify = identify::Behaviour::new(/* ... */);

let peer_manager = PeerManager::new(/* ... */)?;

let connection_limits = /* ... */;

let upnp = /* ... */;

// 8. Behaviour 구성

let behaviour = Behaviour {

gossipsub,

eth2_rpc,

discovery,

identify,

peer_manager,

connection_limits,

upnp,

};

// 9. Swarm 생성

let swarm = SwarmBuilder::with_existing_identity(local_keypair)

.with_tokio()
```

```
.with_other_transport(|_key| transport)
.with_behaviour(|_| behaviour)
.with_swarm_config(|_| config)
.build();
// 10. Network 구조체 생성
let mut network = Network {
swarm,
network_globals,
enr_fork_id,
network_dir: config.network_dir.clone(),
fork_context: ctx.fork_context,
score_settings,
update_gossipsub_scores,
gossip_cache,
local_peer_id,
};
// 11. 네트워크 시작
network.start(&config).await?;
Ok((network, network.network_globals.clone()))
}
}
```

네트워크 시작 과정

```
async fn start(&mut self, config: &crate::NetworkConfig) -> Result<(), String

// 1. 리스닝 주소 설정

for listen_multiaddr in config.listen_addrs().libp2p_addresses() {

// QUIC 비활성화 시 QUIC 포트 무시

if config.disable_quic_support

&& listen_multiaddr.iter().any(|v| v == MProtocol::QuicV1)

{

continue;

}



match self.swarm.listen_on(listen_multiaddr.clone()) {

Ok(_) => {

let mut log_address = listen_multiaddr;

log_address.push(MProtocol::P2p(enr.peer_id()));

info!(address = %log_address, "Listening established");

}

Err(err) => {

return Err("Libp2p was unable to listen on the given listen address.".into());

}

};

}
```

```
// 2. 부트노드 연결
for bootnode_enr in boot_nodes {
// QUIC 연결 시도 (활성화된 경우)
if !config.disable_quic_support {
for quic_multiaddr in &bootnode_enr.multiaddr_quic() {
if !self.network_globals.peers.read()
.is_connected_or_dialing(&bootnode_enr.peer_id())
{
dial(quic_multiaddr.clone());
}
}
}

// TCP 연결 시도
for multiaddr in &bootnode_enr.multiaddr() {
// UDP 주소 무시
let components = multiaddr.iter().collect::<Vec<_>>();
if let MProtocol::Udp(_) = components[1] {
continue;
}

if !self.network_globals.peers.read()
.is_connected_or_dialing(&bootnode_enr.peer_id())
```

```
{
dial(multiaddr.clone());
}
}
}

// 3. 토픽 구독
let mut subscribed_topics: Vec<GossipKind> = vec![];
for topic_kind in &config.topics {
if self.subscribe_kind(topic_kind.clone()) {
subscribed_topics.push(topic_kind.clone());
} else {
warn!(topic = %topic_kind, "Could not subscribe to topic");
}
}

if !subscribed_topics.is_empty() {
info!(topics = ?subscribed_topics, "Subscribed to topics");
}

Ok(())
}
```

메시지 처리 흐름

Gossipsub 메시지 수신 및 처리

1. **메시지 수신**: libp2p Gossipsub에서 메시지 수신
2. **토픽 디코딩**: 토픽 해시에서 GossipTopic 추출
3. **메시지 검증**: 기본적인 형식 및 크기 검증
4. **압축 해제**: Snappy 압축 해제
5. **SSZ 디코딩**: 메시지 내용을 적절한 타입으로 디코딩
6. **상위 레이어 전달**: BeaconProcessor로 메시지 전달

RPC 요청/응답 처리

1. **요청 수신**: 피어로부터 RPC 요청 수신
2. **프로토콜 협상**: 적절한 RPC 프로토콜 버전 협상
3. **요청 처리**: 요청 타입에 따른 적절한 핸들러 호출
4. **응답 생성**: 요청에 대한 응답 데이터 생성
5. **응답 전송**: 스트리밍 방식으로 응답 전송

성능 최적화 및 보안

피어 스코어링을 통한 품질 관리

```
// Gossipsub 피어 스코어 매개변수
let  params = score_settings.get_peer_score_params(
active_validators,
```

```
&thresholds,
&enr_fork_id,
current_slot,
)?;


// 스코어 기반 피어 관리
gossipsub.with_peer_score(params, thresholds)?;
```

연결 제한 및 리소스 관리

- **최대 연결 수 제한**: 메모리 및 CPU 사용량 제어
- **서브스트림 제한**: 동시 RPC 요청 수 제한
- **대역폭 모니터링**: 네트워크 사용량 추적
- **피어 다양성**: 지리적 및 네트워크 다양성 확보

보안 메커니즘

1. **피어 평판 시스템**: 악의적 행동 감지 및 차단
2. **연결 암호화**: Noise 프로토콜을 통한 안전한 통신
3. **메시지 검증**: 모든 수신 메시지의 무결성 검증
4. **DoS 방지**: 속도 제한 및 리소스 제한

결론

Lighthouse의 네트워킹 시스템은 Ethereum 2.0의 분산 합의를 위한 견고하고 효율적인 P2P 통신 인프라를 제공합니다. Gossipsub을 통한 효율적인 메시지 전파, RPC를 통한 직접적인 피어 통신, 그리고 정교한 피어 관리 시스템이 결합되어 안전하고 확장 가능한 네트워크를 구성합니다.

이 시스템의 핵심 강점은 다음과 같습니다:

1. **모듈화된 설계**: 각 컴포넌트가 독립적으로 동작하면서도 효과적으로 협력
2. **적응적 피어 관리**: 네트워크 상황에 따른 동적 피어 선택 및 관리
3. **효율적인 메시지 전파**: Gossipsub을 통한 최적화된 메시지 라우팅
4. **강력한 보안**: 다층적 보안 메커니즘을 통한 네트워크 보호
5. **확장성**: 대규모 검증자 네트워크를 지원하는 확장 가능한 아키텍처

이러한 특징들이 결합되어 Lighthouse는 Ethereum 2.0 네트워크에서 안정적이고 효율적인 합의 참여를 가능하게 합니다.